GCIS-124 Practicum 3 Makeup

Please see the README.txt in your repository before beginning any solutions.

This practical consists of 3 questions for a total of 80 points. Note that some questions have multiple parts. The total points for each question and part are below. You may attempt these in any order and can move to another question if stuck.

For question 3, choose one of 3a or 3b.

You may only use Java constructs and features covered in Units 1-10. If you're unsure whether something is allowed, ask — it's better than losing points.

In fact, ask if you have any questions at all.

You are expected to name your classes, attributes, and methods exactly as given when specified. If you deviate from the identifiers given, points will be deducted.

Provide only what is asked for. Do not invent requirements. No question on the exam requires user input.

Follow the instructions closely. Points will be deducted for not following instructions.

Question 1 (20 total points, 1 part)

All work for this question will go in the practicum.q1 package.

You have a broken music player that plays a random number of songs at the same time for a random amount of time each.

In the BrokenMusicPlayer.java file, do the following:

- main should create and start a random number of threads between 2 and 8 (inclusive) - use your preferred method to create and start threads.
- Each thread should do the following:
  - Select *one* random song from SONGS.
  - Randomly select a play time between 100 and 2000 ms (inclusive).
  - Print *exactly* the following: "Playing <song> for <play time> ms".
  - Pause the thread the amount of play time to simulate playing the song.
  - Print *exactly* the following: "Done playing <song>"
- No thread synchronization is necessary.
- It is ok if a song is selected by multiple threads - your music player is broken after all.

Example:

```
Playing Hallelujah for 1159 ms
Playing Purple Haze for 1476 ms
Playing Livin' on a Prayer for 1004 ms
Playing Purple Haze for 111 ms
Playing Born to Run for 1036 ms
Playing Highway to Hell for 1027 ms
Playing Free Bird for 1006 ms
Done playing Purple Haze
Done playing Livin' on a Prayer
Done playing Free Bird
Done playing Highway to Hell
Done playing Born to Run
Done playing Hallelujah
Done playing Purple Haze
```

Question 2 (30 total points, 1 part, 2 bonus)

All work for this question will go in the **practicum.q2** package.

MaxFinder
- name: String - values: List - max: Integer
+ <<create>> MaxFinder(name: String, values: List) + findMax(): void + getMax(): Integer + toString(): String

Examine the **MaxFinder** UML above and **MaxFinder.java**. An instance is created with a **name** and an **Integer List**. When **findMax()** is called, the method will find the maximum value in the list and store it in the **max** attribute. Raise your hand if you have any questions.

Inside the **main** method of the **ParallelMax.java** file is a list of **MaxFinder**s.

The **main()** method should:

- Start all **MaxFinder**s finding at the same time and in parallel.
- After all **MaxFinder**s have started finding, print **"All MaxFinders have started finding."**
- After all **MaxFinders** have finished finding:
    - Print **"All MaxFinders have finished finding."**
    - Print the string representation of each **MaxFinder** on a separate line.
    - Collect all of the max values from each **MaxFinder** into a new **Integer List**.
    - Create a *new* **MaxFinder** to find and print the overall max value.
        - This should run in the main thread.

You *may* update **MaxFinder** to implement an interface *if necessary*.

See **example_output.txt** for expected output format.

Questions 3a and 3b are both client/server network questions. Choose one of the two.

Question 3a (30 total points, 1 part):

All work for this question will go in the **practicum.q3a** package. For this question, you will be writing a **TCP/IP client** that collects one of each item in a collection.

The **Duplexer** class has been provided, but you are not required to use it.

Two ways to test your client against a server:

1. Locally
    a. Open a terminal
    b. Run the following command: **java -jar collector-server.jar**
    c. Connect to **localhost:20012**
    d. This is preferred as you will be able to see the server output
2. Remotely
    - Connect to **holly.cs.rit.edu:20012**

Requirements:

Inside **CollectorClient.java**.

- Update the **static collectItems** method that takes a **Socket** as its only parameter, has a **void** return type, to do the following:
    - Send the message **"START"** to the server.
    - The server will respond with the number of unique items in the collection (an integer).
      Integer.parseInt(<int>) will convert a string into an integer.
    - Send the message **"GO"**
    - The *server* will then send a series of string messages, where each message is an item in the collection. Duplicate items are likely to be sent, but remember, your client wants to collect one of each item.
        - After each item is sent, your client should respond with the number of unique items collected thus far.
            - That is, the client should ignore any duplicate items received in its count.

          Integer.toString(<int>) will convert an int into a string
        - You do not know what items or the order in which the items will be sent, so don't make any assumptions in this regard.
        - For full credit, keeping track of unique items from the collection must be an **O(1)** operation.
    - Once the client has collected all of the unique items in the collection and sent that count back, the server will send back **"GOODBYE"** and the client can exit.
    - Your client should print out all messages to and from the server. See next page for example of proper output format and data flow.
    - Be sure to close the socket.

(Continued on next page)

Question 3a (continued)

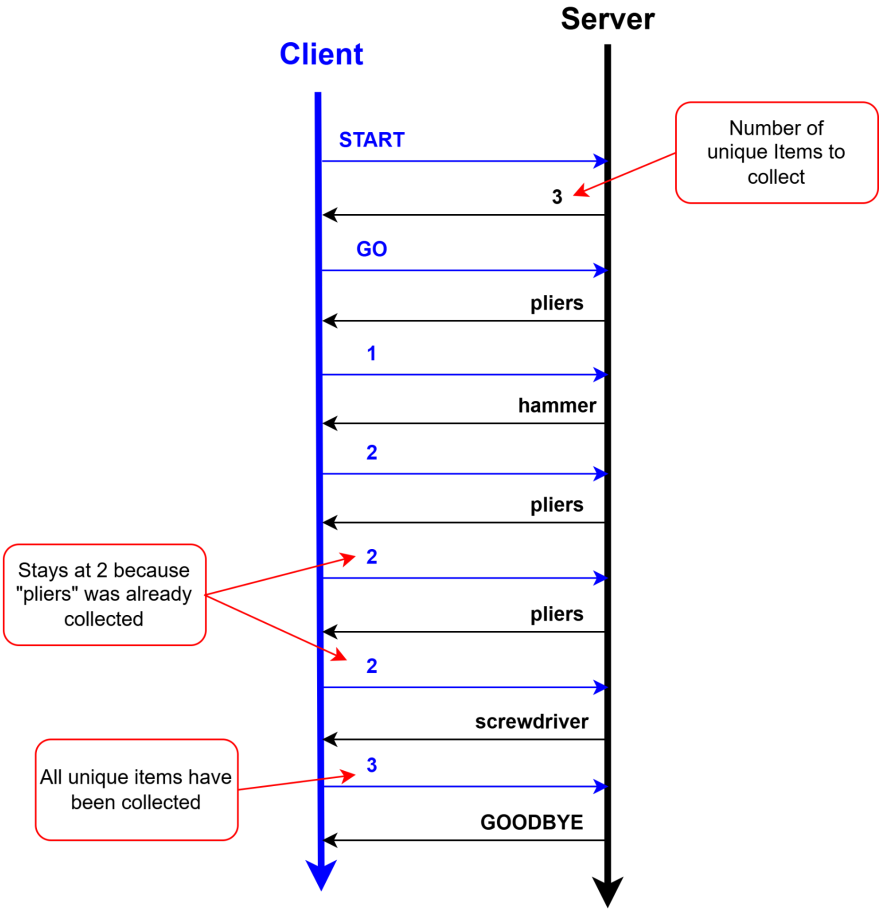
Requirements: (continued)

- Define a **main** that does the following:
  - Connects to the server.
  - Calls **collectItems** with the socket.

Tests for **collectItems** have been provided in **CollectorClientTest.txt**. Rename to **.java**.

Example Output and Data Flow

```
CONNECTED TO: localhost:20012
SENT: START
RECEIVED: 3
SENT: GO
RECEIVED: pliers
SENT: 1
RECEIVED: hammer
SENT: 2
RECEIVED: pliers
SENT: 2
RECEIVED: pliers
SENT: 2
RECEIVED: screwdriver
SENT: 3
RECEIVED: GOODBYE
```



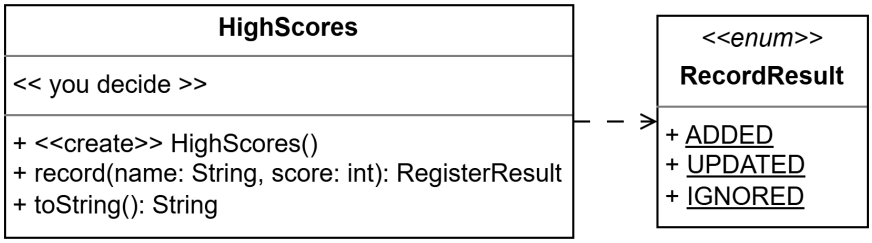
Questions 3a and 3b are both client/server network questions. Choose one of the two.

Question 3b (30 total points, 3 parts):

All work for this question will go in the **practicum.q3b** package. For this question, you will be writing an **HTTP server** that keeps track of people's names and their high scores.

You have been provided with the **httpserver** package.

Part A (5 points)



Update the **HighScores** class according to the following requirements:

- **RecordResult** has been provided as an inner enum.
- You need to decide what data structure(s) will be used for storing names (strings) and associated scores (integers) based on the rest of the requirements.
- You may assume all **name**s are one word (no spaces) and are unique.
- **record()**
  - If the given **name** is not already in **HighScores**, the **name** and **score** are added and **ADDED** is returned.
  - If the **name** is in **HighScores**:
    - If the given **score** is less than or equal to the existing recorded score, no update is made and **IGNORED** is returned.
    - If the given **score** is greater than the existing recorded score, the score is updated and **UPDATED** is returned.
- **toString()**
  - Returns the contents of HighScores with names in alphabetical order, in this exact format:
    <name 1> <high score 1>|<name 2> <high score 2>|...|<name N> <high score N>
    For example:
    Alice 50|Bob 45|Fred 32
    - If no names have been recorded, an empty string, i.e. "", should be returned.
  - **String.join("|", <collection>)** works with most collections of Strings: arrays, lists, sets, map.keySet(), etc.
- For full credit, both methods must have a time complexity of **O(n)** *or better*.

Tests have been provided in **HighScores.txt**. Rename to **.java**.

See next page for **Part**s **B** and **C**.

Question 3b Part B (20 points)

Create a **class** named **HighScoreHandler** that implements **RequestHandler** and handles HTTP requests from a client.

Protocol

Request			Response	
Method	URI	Body	Status	Body
GET	/	N/A	200 OK	<name 1> <high score 1>|<name 2> <high score 2>|...|<name N> <high score N> Alice 50|Bob 45|Fred 32
POST	/	<name> <high score> Bob 50	200 OK	ADDED UPDATED IGNORED

Requirements

- Use your **HighScores** class from **Part A**.
- When the server receives a **POST /** request, it should record the name and score with **HighScores** and return the result as the response body. You will have to convert the **RecordResult** enum to a string.
- When the server receives a **GET /** request, a string representing the contents of **HighScores** should be returned

Tests have been provided in **HighScoreHandlerTest.txt**. Rename to **.java**.

Part C (5 points)

Create and start an HTTP Server named **HighScoreServer**. The server must listen for clients on port **8081**.

Start the server and use the provided **high-score-client.html** to test. See the **html_client_screenshots/*.png** files for example client displays.